

JAVA LAB RECORD

Collection Framework Methods – All Programs

Name: S.Trivikram

Roll Number: A25126552251

Date: 21/09/26

All supplied programs are arranged in the requested lab-record format.

Experiment No: 01

Basic Information

Experiment No	01
Date	21/09/26
Title	Implementation of Collection Interface Methods in Java
Name	S.Trivikram
Roll Number	A25126552251

1. Objective / Aim

To implement and demonstrate commonly used methods of the Java Collection interface using an ArrayList, including add(), addAll(), contains(), containsAll(), size(), isEmpty(), remove(), removeAll(), iterator(), and clear().

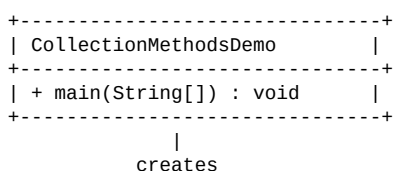
2. System Requirements

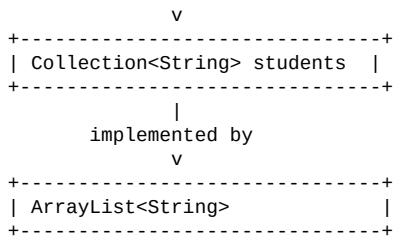
Hardware Requirement	Specification
Processor	Intel Core i3 or equivalent and above
RAM	Minimum 4 GB
Storage	Minimum 1 GB free space
Input Device	Keyboard and mouse
Display	Standard monitor
Software Requirement	Specification
Operating System	Windows / Linux / macOS
Java	JDK 17 or JDK 21
IDE / Editor	Eclipse / IntelliJ IDEA / NetBeans / VS Code
Compiler	javac
Execution Environment	Java Virtual Machine (JVM)

3. Algorithm / Logic Description

1. Start the program.
2. Import ArrayList, Arrays, Collection, and Iterator classes.
3. Create a Collection named students using ArrayList.
4. Add initial student names using add().
5. Create another collection using Arrays.asList() and add its elements using addAll().
6. Use contains() and containsAll() to check elements.
7. Display size and check isEmpty().
8. Remove elements using remove() and removeAll().
9. Traverse remaining elements using Iterator, hasNext(), and next().
10. Use clear() and verify the collection is empty.
11. Stop the program.

4. UML Class Diagram





5. Source Code

```

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collection;
import java.util.Iterator;

class CollectionMethodsDemo {
    public static void main(String[] args) {
        Collection<String> students = new ArrayList<>();

        students.add("Rahul");
        students.add("Priya");
        students.add("Arjun");
        System.out.println("After add(): " + students);

        Collection<String> newStudents =
            Arrays.asList("Anu", "Kiran");
        students.addAll(newStudents);
        System.out.println("After addAll(): " + students);

        System.out.println("Contains Rahul? "
            + students.contains("Rahul"));
        System.out.println("Contains Anu and Kiran? "
            + students.containsAll(newStudents));

        System.out.println("Size: " + students.size());
        System.out.println("Is collection empty? "
            + students.isEmpty());

        students.remove("Arjun");
        System.out.println("After remove(): " + students);

        students.removeAll(newStudents);
        System.out.println("After removeAll(): " + students);

        System.out.println("Using iterator():");
        Iterator<String> iterator = students.iterator();
        while (iterator.hasNext()) {
            System.out.println(iterator.next());
        }

        students.clear();
        System.out.println("After clear(): " + students);
        System.out.println("Is collection empty now? "
            + students.isEmpty());
    }
}

```

6. Completion and Execution Commands

Step 1: Save the program as:
CollectionMethodsDemo.java

Step 2: Compile the program:
javac CollectionMethodsDemo.java

Step 3: Run the program:
java CollectionMethodsDemo

7. Output

```

After add(): [Rahul, Priya, Arjun]
After addAll(): [Rahul, Priya, Arjun, Anu, Kiran]
Contains Rahul? true
Contains Anu and Kiran? true
Size: 5
Is collection empty? false
After remove(): [Rahul, Priya, Anu, Kiran]
After removeAll(): [Rahul, Priya]
Using iterator():
Rahul

```

```
Priya  
After clear(): []  
Is collection empty now? true
```

Result: The Java program was successfully implemented and the specified collection framework methods were demonstrated.

Experiment No: 02

Basic Information

Experiment No	02
Date	21/09/26
Title	Implementation of Deque Interface Methods in Java
Name	S.Trivikram
Roll Number	A25126552251

1. Objective / Aim

To implement and demonstrate common Deque methods for insertion, inspection, and removal from both ends using ArrayDeque.

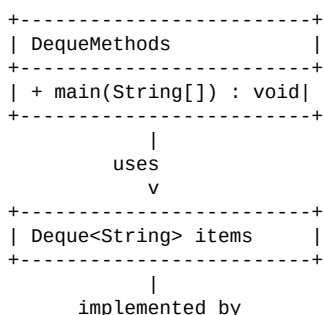
2. System Requirements

Hardware Requirement	Specification
Processor	Intel Core i3 or equivalent and above
RAM	Minimum 4 GB
Storage	Minimum 1 GB free space
Input Device	Keyboard and mouse
Display	Standard monitor
Software Requirement	Specification
Operating System	Windows / Linux / macOS
Java	JDK 17 or JDK 21
IDE / Editor	Eclipse / IntelliJ IDEA / NetBeans / VS Code
Compiler	javac
Execution Environment	Java Virtual Machine (JVM)

3. Algorithm / Logic Description

1. Start the program.
2. Import Deque and ArrayDeque.
3. Create a Deque of strings.
4. Insert elements using addFirst(), addLast(), offerFirst(), and offerLast().
5. Inspect the first and last elements using peekFirst() and peekLast().
6. Remove elements from both ends using removeFirst(), removeLast(), pollFirst(), and pollLast().
7. Display the deque after each operation.
8. Stop the program.

4. UML Class Diagram



```

      v
+-----+
| ArrayDeque<String> |
+-----+

```

5. Source Code

```

import java.util.Deque;
import java.util.ArrayDeque;

class DequeMethods {
    public static void main(String[] args) {
        Deque<String> items = new ArrayDeque<>();

        items.addFirst("Book");
        items.addFirst("Pen");
        System.out.println("After addFirst(): " + items);

        items.addLast("Bag");
        items.addLast("Bottle");
        System.out.println("After addLast(): " + items);

        items.offerFirst("Notebook");
        System.out.println("After offerFirst(): " + items);

        items.offerLast("Pencil");
        System.out.println("After offerLast(): " + items);

        System.out.println("First element: " + items.peekFirst());
        System.out.println("Last element: " + items.peekLast());

        System.out.println("Removed first: " + items.removeFirst());
        System.out.println("After removeFirst(): " + items);

        System.out.println("Removed last: " + items.removeLast());
        System.out.println("After removeLast(): " + items);

        System.out.println("Polled first: " + items.pollFirst());
        System.out.println("After pollFirst(): " + items);

        System.out.println("Polled last: " + items.pollLast());
        System.out.println("After pollLast(): " + items);
    }
}

```

6. Completion and Execution Commands

Step 1: Save the program as:
DequeMethods.java

Step 2: Compile the program:
javac DequeMethods.java

Step 3: Run the program:
java DequeMethods

7. Output

```

After addFirst(): [Pen, Book]
After addLast(): [Pen, Book, Bag, Bottle]
After offerFirst(): [Notebook, Pen, Book, Bag, Bottle]
After offerLast(): [Notebook, Pen, Book, Bag, Bottle, Pencil]
First element: Notebook
Last element: Pencil
Removed first: Notebook
After removeFirst(): [Pen, Book, Bag, Bottle, Pencil]
Removed last: Pencil
After removeLast(): [Pen, Book, Bag, Bottle]
Polled first: Pen
After pollFirst(): [Book, Bag, Bottle]
Polled last: Bottle
After pollLast(): [Book, Bag]

```

Result: The Java program was successfully implemented and the specified collection framework methods were demonstrated.

Experiment No: 03

Basic Information

Experiment No	03
Date	21/09/26
Title	Implementation of Iterator Interface Methods in Java
Name	S.Trivikram
Roll Number	A25126552251

1. Objective / Aim

To demonstrate Iterator methods hasNext(), next(), remove(), and forEachRemaining() for traversing and modifying an ArrayList.

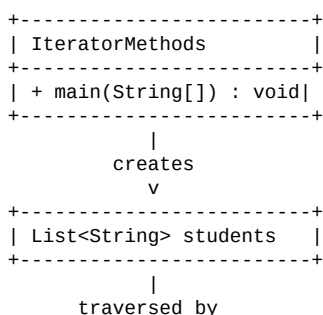
2. System Requirements

Hardware Requirement	Specification
Processor	Intel Core i3 or equivalent and above
RAM	Minimum 4 GB
Storage	Minimum 1 GB free space
Input Device	Keyboard and mouse
Display	Standard monitor
Software Requirement	Specification
Operating System	Windows / Linux / macOS
Java	JDK 17 or JDK 21
IDE / Editor	Eclipse / IntelliJ IDEA / NetBeans / VS Code
Compiler	javac
Execution Environment	Java Virtual Machine (JVM)

3. Algorithm / Logic Description

1. Start the program.
2. Import ArrayList, Iterator, and List.
3. Create a list of student names.
4. Create an Iterator and traverse the list using hasNext() and next().
5. Remove Arjun safely using iterator.remove().
6. Create a new Iterator and display the first element.
7. Use forEachRemaining() to display the remaining elements.
8. Stop the program.

4. UML Class Diagram



```

      v
+-----+
| Iterator<String> |
+-----+

```

5. Source Code

```

import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

class IteratorMethods {
    public static void main(String[] args) {
        List<String> students = new ArrayList<>();

        students.add("Rahul");
        students.add("Priya");
        students.add("Arjun");
        students.add("Meera");
        students.add("Anjali");

        System.out.println("Original List: " + students);

        Iterator<String> iterator = students.iterator();

        System.out.println("\nUsing hasNext() and next():");
        while (iterator.hasNext()) {
            String student = iterator.next();
            System.out.println("Student: " + student);

            if (student.equals("Arjun")) {
                iterator.remove();
                System.out.println("Arjun removed.");
            }
        }

        System.out.println("\nList after remove(): " + students);

        iterator = students.iterator();
        if (iterator.hasNext()) {
            System.out.println("\nFirst element: " + iterator.next());
        }

        System.out.println("\nRemaining elements using forEachRemaining():");
        iterator.forEachRemaining(student -> {
            System.out.println("Student: " + student);
        });
    }
}

```

6. Completion and Execution Commands

Step 1: Save the program as:
IteratorMethods.java

Step 2: Compile the program:
javac IteratorMethods.java

Step 3: Run the program:
java IteratorMethods

7. Output

Original List: [Rahul, Priya, Arjun, Meera, Anjali]

Using hasNext() and next():

```

Student: Rahul
Student: Priya
Student: Arjun
Arjun removed.
Student: Meera
Student: Anjali

```

List after remove(): [Rahul, Priya, Meera, Anjali]

First element: Rahul

Remaining elements using forEachRemaining():

```

Student: Priya
Student: Meera
Student: Anjali

```

Result: The Java program was successfully implemented and the specified collection framework methods were demonstrated.

Experiment No: 04

Basic Information

Experiment No	04
Date	21/09/26
Title	Implementation of List Interface Methods in Java
Name	S.Trivikram
Roll Number	A25126552251

1. Objective / Aim

To implement and demonstrate commonly used List methods such as add(), get(), set(), remove(), indexOf(), lastIndexOf(), subList(), and sort().

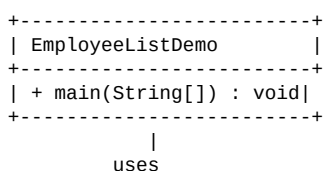
2. System Requirements

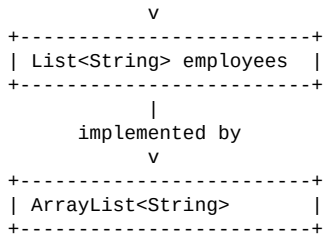
Hardware Requirement	Specification
Processor	Intel Core i3 or equivalent and above
RAM	Minimum 4 GB
Storage	Minimum 1 GB free space
Input Device	Keyboard and mouse
Display	Standard monitor
Software Requirement	Specification
Operating System	Windows / Linux / macOS
Java	JDK 17 or JDK 21
IDE / Editor	Eclipse / IntelliJ IDEA / NetBeans / VS Code
Compiler	javac
Execution Environment	Java Virtual Machine (JVM)

3. Algorithm / Logic Description

1. Start the program.
2. Import ArrayList, Comparator, and List.
3. Create an employee list and add employee names.
4. Insert an employee at a specified index using add(index, element).
5. Retrieve an element using get().
6. Replace an element using set().
7. Remove an element using remove(index).
8. Find first and last positions using indexOf() and lastIndexOf().
9. Create a portion using subList().
10. Sort the list using Comparator.naturalOrder().
11. Stop the program.

4. UML Class Diagram





5. Source Code

```

import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

class EmployeeListDemo {
    public static void main(String[] args) {
        List<String> employees = new ArrayList<>();

        employees.add("Anjali");
        employees.add("Vikram");
        employees.add("Rohan");
        employees.add("Anjali");
        System.out.println("Original List: " + employees);

        employees.add(1, "Meena");
        System.out.println("After add(1, \"Meena\"): " + employees);

        System.out.println("Element at index 2: " + employees.get(2));

        employees.set(2, "Karthik");
        System.out.println("After set(2, \"Karthik\"): " + employees);

        employees.remove(3);
        System.out.println("After remove(3): " + employees);

        System.out.println("First index of Anjali: "
            + employees.indexOf("Anjali"));
        System.out.println("Last index of Anjali: "
            + employees.lastIndexOf("Anjali"));

        List<String> selectedEmployees = employees.subList(1, 3);
        System.out.println("SubList from index 1 to 3: "
            + selectedEmployees);

        employees.sort(Comparator.naturalOrder());
        System.out.println("After Sorting: " + employees);
    }
}

```

6. Completion and Execution Commands

Step 1: Save the program as:
EmployeeListDemo.java

Step 2: Compile the program:
javac EmployeeListDemo.java

Step 3: Run the program:
java EmployeeListDemo

7. Output

```

Original List: [Anjali, Vikram, Rohan, Anjali]
After add(1, "Meena"): [Anjali, Meena, Vikram, Rohan, Anjali]
Element at index 2: Vikram
After set(2, "Karthik"): [Anjali, Meena, Karthik, Rohan, Anjali]
After remove(3): [Anjali, Meena, Karthik, Anjali]
First index of Anjali: 0
Last index of Anjali: 3
SubList from index 1 to 3: [Meena, Karthik]
After Sorting: [Anjali, Anjali, Karthik, Meena]

```

Result: The Java program was successfully implemented and the specified collection framework methods were demonstrated.

Experiment No: 05

Basic Information

Experiment No	05
Date	21/09/26
Title	Implementation of ListIterator Methods in Java
Name	S.Trivikram
Roll Number	A25126552251

1. Objective / Aim

To demonstrate bidirectional traversal and modification of a List using ListIterator methods such as hasNext(), next(), nextIndex(), hasPrevious(), previous(), previousIndex(), set(), add(), and remove().

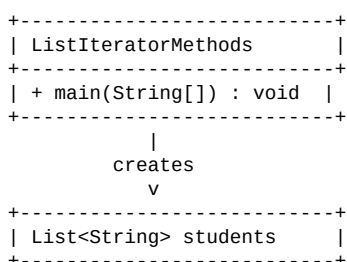
2. System Requirements

Hardware Requirement	Specification
Processor	Intel Core i3 or equivalent and above
RAM	Minimum 4 GB
Storage	Minimum 1 GB free space
Input Device	Keyboard and mouse
Display	Standard monitor
Software Requirement	Specification
Operating System	Windows / Linux / macOS
Java	JDK 17 or JDK 21
IDE / Editor	Eclipse / IntelliJ IDEA / NetBeans / VS Code
Compiler	javac
Execution Environment	Java Virtual Machine (JVM)

3. Algorithm / Logic Description

1. Start the program.
2. Import ArrayList, List, and ListIterator.
3. Create a list of students.
4. Traverse the list forward using hasNext(), next(), and nextIndex().
5. Traverse backward using hasPrevious(), previous(), and previousIndex().
6. Reset the iterator and replace the first element using set().
7. Insert an element using add().
8. Move to the next element and remove it using remove().
9. Display the modified list and stop.

4. UML Class Diagram



```

      |
      v
traversed/modified by
      v
+-----+
| ListIterator<String> |
+-----+

```

5. Source Code

```

import java.util.ArrayList;
import java.util.List;
import java.util.ListIterator;

class ListIteratorMethods {
    public static void main(String[] args) {
        List<String> students = new ArrayList<>();

        students.add("Rahul");
        students.add("Priya");
        students.add("Arjun");
        students.add("Meera");

        System.out.println("Original List: " + students);

        ListIterator<String> iterator = students.listIterator();

        System.out.println("\nForward Traversal:");
        while (iterator.hasNext()) {
            System.out.println(
                "Index: " + iterator.nextIndex() +
                ", Student: " + iterator.next()
            );
        }

        System.out.println("\nBackward Traversal:");
        while (iterator.hasPrevious()) {
            System.out.println(
                "Index: " + iterator.previousIndex() +
                ", Student: " + iterator.previous()
            );
        }

        iterator = students.listIterator();
        iterator.next();
        iterator.set("Aarav");
        System.out.println("\nAfter set(): " + students);

        iterator.add("Diya");
        System.out.println("After add(): " + students);

        iterator.next();
        iterator.remove();
        System.out.println("After remove(): " + students);
    }
}

```

6. Completion and Execution Commands

Step 1: Save the program as:
ListIteratorMethods.java

Step 2: Compile the program:
javac ListIteratorMethods.java

Step 3: Run the program:
java ListIteratorMethods

7. Output

Original List: [Rahul, Priya, Arjun, Meera]

Forward Traversal:
Index: 0, Student: Rahul
Index: 1, Student: Priya
Index: 2, Student: Arjun
Index: 3, Student: Meera

Backward Traversal:
Index: 3, Student: Meera
Index: 2, Student: Arjun
Index: 1, Student: Priya
Index: 0, Student: Rahul

After set(): [Aarav, Priya, Arjun, Meera]
After add(): [Aarav, Diya, Priya, Arjun, Meera]
After remove(): [Aarav, Diya, Arjun, Meera]

Result: The Java program was successfully implemented and the specified collection framework methods were demonstrated.

Experiment No: 06

Basic Information

Experiment No	06
Date	21/09/26
Title	Implementation of Map Interface Methods in Java
Name	S.Trivikram
Roll Number	A25126552251

1. Objective / Aim

To implement and demonstrate commonly used Map methods such as put(), get(), containsKey(), containsValue(), keySet(), values(), entrySet(), size(), isEmpty(), remove(), and clear().

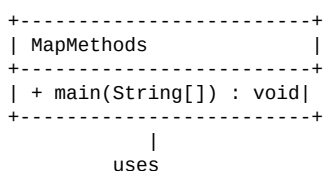
2. System Requirements

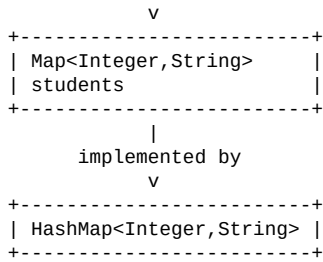
Hardware Requirement	Specification
Processor	Intel Core i3 or equivalent and above
RAM	Minimum 4 GB
Storage	Minimum 1 GB free space
Input Device	Keyboard and mouse
Display	Standard monitor
Software Requirement	Specification
Operating System	Windows / Linux / macOS
Java	JDK 17 or JDK 21
IDE / Editor	Eclipse / IntelliJ IDEA / NetBeans / VS Code
Compiler	javac
Execution Environment	Java Virtual Machine (JVM)

3. Algorithm / Logic Description

1. Start the program.
2. Import Map and HashMap.
3. Create a HashMap for student IDs and names.
4. Add key-value pairs using put().
5. Retrieve a value using get().
6. Check keys and values using containsKey() and containsValue().
7. Display keys, values, and entries.
8. Display the map size and empty status.
9. Remove one entry using remove().
10. Clear the map and verify that it is empty.
11. Stop the program.

4. UML Class Diagram





5. Source Code

```

import java.util.Map;
import java.util.HashMap;

class MapMethods {
    public static void main(String[] args) {
        Map<Integer, String> students = new HashMap<>();

        students.put(101, "Aarav");
        students.put(102, "Meera");
        students.put(103, "Rohan");
        students.put(104, "Diya");

        System.out.println("Map after put(): " + students);
        System.out.println("Value for key 102: " + students.get(102));

        System.out.println("Contains key 103: "
            + students.containsKey(103));
        System.out.println("Contains value Rohan: "
            + students.containsValue("Rohan"));

        System.out.println("Keys: " + students.keySet());
        System.out.println("Values: " + students.values());
        System.out.println("Entries: " + students.entrySet());

        System.out.println("Map size: " + students.size());
        System.out.println("Is map empty: " + students.isEmpty());

        students.remove(104);
        System.out.println("After remove(104): " + students);

        students.clear();
        System.out.println("After clear(): " + students);
        System.out.println("Is map empty after clear: "
            + students.isEmpty());
    }
}

```

6. Completion and Execution Commands

Step 1: Save the program as:
MapMethods.java

Step 2: Compile the program:
javac MapMethods.java

Step 3: Run the program:
java MapMethods

7. Output

```

Map after put(): {101=Aarav, 102=Meera, 103=Rohan, 104=Diya}
Value for key 102: Meera
Contains key 103: true
Contains value Rohan: true
Keys: [101, 102, 103, 104]
Values: [Aarav, Meera, Rohan, Diya]
Entries: [101=Aarav, 102=Meera, 103=Rohan, 104=Diya]
Map size: 4
Is map empty: false
After remove(104): {101=Aarav, 102=Meera, 103=Rohan}
After clear(): {}
Is map empty after clear: true

```

Note: HashMap does not guarantee iteration order; displayed map/key order may vary.

Result: The Java program was successfully implemented and the specified collection framework methods were demonstrated.

Experiment No: 07

Basic Information

Experiment No	07
Date	21/09/26
Title	Implementation of NavigableSet Interface Methods in Java
Name	S.Trivikram
Roll Number	A25126552251

1. Objective / Aim

To implement and demonstrate NavigableSet methods such as lower(), floor(), ceiling(), higher(), pollFirst(), pollLast(), and descendingSet() using TreeSet.

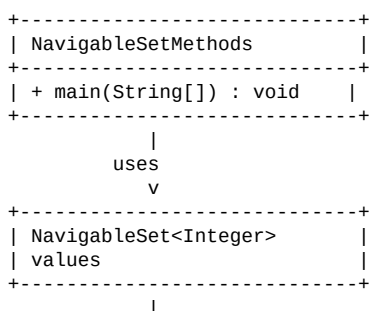
2. System Requirements

Hardware Requirement	Specification
Processor	Intel Core i3 or equivalent and above
RAM	Minimum 4 GB
Storage	Minimum 1 GB free space
Input Device	Keyboard and mouse
Display	Standard monitor
Software Requirement	Specification
Operating System	Windows / Linux / macOS
Java	JDK 17 or JDK 21
IDE / Editor	Eclipse / IntelliJ IDEA / NetBeans / VS Code
Compiler	javac
Execution Environment	Java Virtual Machine (JVM)

3. Algorithm / Logic Description

1. Start the program.
2. Import NavigableSet and TreeSet.
3. Create a NavigableSet using TreeSet and add integer values.
4. Use lower() and floor() to find values below or equal to a specified value.
5. Use ceiling() and higher() to find values equal to or above a specified value.
6. Remove the first and last elements using pollFirst() and pollLast().
7. Display the set in descending order using descendingSet().
8. Stop the program.

4. UML Class Diagram



```

        implemented by
          v
+-----+
| TreeSet<Integer> |
+-----+

```

5. Source Code

```

import java.util.NavigableSet;
import java.util.TreeSet;

class NavigableSetMethods {
    public static void main(String[] args) {
        NavigableSet<Integer> values = new TreeSet<>();

        values.add(10);
        values.add(20);
        values.add(30);
        values.add(40);
        values.add(50);
        values.add(60);

        System.out.println("NavigableSet: " + values);
        System.out.println("Lower than 30: " + values.lower(30));
        System.out.println("Floor of 30: " + values.floor(30));
        System.out.println("Ceiling of 35: " + values.ceiling(35));
        System.out.println("Higher than 30: " + values.higher(30));

        System.out.println("Removed first element: "
            + values.pollFirst());
        System.out.println("After pollFirst(): " + values);

        System.out.println("Removed last element: "
            + values.pollLast());
        System.out.println("After pollLast(): " + values);

        System.out.println("Descending Set: " + values.descendingSet());
    }
}

```

6. Completion and Execution Commands

Step 1: Save the program as:
NavigableSetMethods.java

Step 2: Compile the program:
javac NavigableSetMethods.java

Step 3: Run the program:
java NavigableSetMethods

7. Output

```

NavigableSet: [10, 20, 30, 40, 50, 60]
Lower than 30: 20
Floor of 30: 30
Ceiling of 35: 40
Higher than 30: 40
Removed first element: 10
After pollFirst(): [20, 30, 40, 50, 60]
Removed last element: 60
After pollLast(): [20, 30, 40, 50]
Descending Set: [50, 40, 30, 20]

```

Result: The Java program was successfully implemented and the specified collection framework methods were demonstrated.

Experiment No: 08

Basic Information

Experiment No	08
Date	21/09/26
Title	Implementation of NavigableMap Interface Methods in Java
Name	S.Trivikram
Roll Number	A25126552251

1. Objective / Aim

To implement and demonstrate NavigableMap methods such as lowerKey(), floorKey(), ceilingKey(), higherKey(), firstEntry(), lastEntry(), pollFirstEntry(), pollLastEntry(), and descendingMap() using TreeMap.

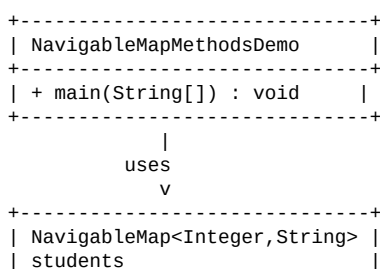
2. System Requirements

Hardware Requirement	Specification
Processor	Intel Core i3 or equivalent and above
RAM	Minimum 4 GB
Storage	Minimum 1 GB free space
Input Device	Keyboard and mouse
Display	Standard monitor
Software Requirement	Specification
Operating System	Windows / Linux / macOS
Java	JDK 17 or JDK 21
IDE / Editor	Eclipse / IntelliJ IDEA / NetBeans / VS Code
Compiler	javac
Execution Environment	Java Virtual Machine (JVM)

3. Algorithm / Logic Description

1. Start the program.
2. Import NavigableMap and TreeMap.
3. Create a NavigableMap of student IDs and names.
4. Add key-value pairs using put().
5. Use lowerKey(), floorKey(), ceilingKey(), and higherKey() to navigate keys.
6. Display firstEntry() and lastEntry().
7. Remove the first and last entries using pollFirstEntry() and pollLastEntry().
8. Display the remaining map in descending order using descendingMap().
9. Stop the program.

4. UML Class Diagram



5. Source Code

Experiment No: 09

Basic Information

Experiment No	09
Date	21/09/26
Title	Implementation of Queue Interface Methods in Java
Name	S.Trivikram
Roll Number	A25126552251

1. Objective / Aim

To implement and demonstrate Queue methods add(), offer(), element(), peek(), remove(), and poll() using LinkedList.

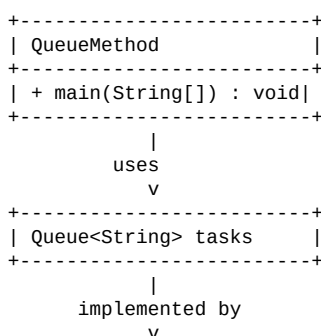
2. System Requirements

Hardware Requirement	Specification
Processor	Intel Core i3 or equivalent and above
RAM	Minimum 4 GB
Storage	Minimum 1 GB free space
Input Device	Keyboard and mouse
Display	Standard monitor
Software Requirement	Specification
Operating System	Windows / Linux / macOS
Java	JDK 17 or JDK 21
IDE / Editor	Eclipse / IntelliJ IDEA / NetBeans / VS Code
Compiler	javac
Execution Environment	Java Virtual Machine (JVM)

3. Algorithm / Logic Description

1. Start the program.
2. Import Queue and LinkedList.
3. Create a Queue of tasks using LinkedList.
4. Insert tasks using add() and offer().
5. Inspect the head using element() and peek().
6. Remove elements using remove() and poll().
7. Display the queue after each operation.
8. Stop the program.

4. UML Class Diagram



```
+-----+
| LinkedList<String> |
+-----+
```

5. Source Code

```
import java.util.Queue;
import java.util.LinkedList;

class QueueMethod {
    public static void main(String[] args) {
        Queue<String> tasks = new LinkedList<>();

        tasks.add("Study");
        tasks.add("Practice");
        tasks.add("Assignment");
        System.out.println("Queue after add(): " + tasks);

        tasks.offer("Project");
        System.out.println("Queue after offer(): " + tasks);

        System.out.println("Head using element(): "
            + tasks.element());
        System.out.println("Head using peek(): "
            + tasks.peek());

        System.out.println("Removed using remove(): "
            + tasks.remove());
        System.out.println("Queue after remove(): " + tasks);

        System.out.println("Removed using poll(): "
            + tasks.poll());
        System.out.println("Queue after poll(): " + tasks);
    }
}
```

6. Completion and Execution Commands

Step 1: Save the program as:
QueueMethod.java

Step 2: Compile the program:
javac QueueMethod.java

Step 3: Run the program:
java QueueMethod

7. Output

```
Queue after add(): [Study, Practice, Assignment]
Queue after offer(): [Study, Practice, Assignment, Project]
Head using element(): Study
Head using peek(): Study
Removed using remove(): Study
Queue after remove(): [Practice, Assignment, Project]
Removed using poll(): Practice
Queue after poll(): [Assignment, Project]
```

Result: The Java program was successfully implemented and the specified collection framework methods were demonstrated.

Experiment No: 10

Basic Information

Experiment No	10
Date	21/09/26
Title	Implementation of Set Interface Methods in Java
Name	S.Trivikram
Roll Number	A25126552251

1. Objective / Aim

To implement and demonstrate Set methods add(), remove(), contains(), size(), isEmpty(), and clear(), and show that duplicate values are not stored in a HashSet.

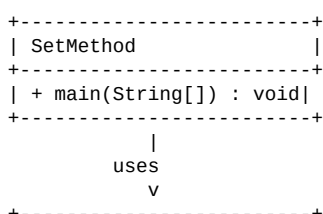
2. System Requirements

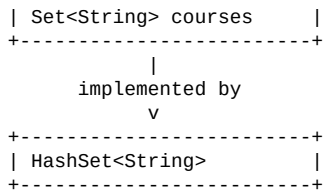
Hardware Requirement	Specification
Processor	Intel Core i3 or equivalent and above
RAM	Minimum 4 GB
Storage	Minimum 1 GB free space
Input Device	Keyboard and mouse
Display	Standard monitor
Software Requirement	Specification
Operating System	Windows / Linux / macOS
Java	JDK 17 or JDK 21
IDE / Editor	Eclipse / IntelliJ IDEA / NetBeans / VS Code
Compiler	javac
Execution Environment	Java Virtual Machine (JVM)

3. Algorithm / Logic Description

1. Start the program.
2. Import Set and HashSet.
3. Create a Set of course names.
4. Add course names using add(), including a duplicate value.
5. Display the set and observe that the duplicate is not added.
6. Remove an element using remove().
7. Check membership using contains().
8. Display size and empty status.
9. Clear the set and verify that it is empty.
10. Stop the program.

4. UML Class Diagram





5. Source Code

```

import java.util.HashSet;
import java.util.Set;

class SetMethod {
    public static void main(String[] args) {
        Set<String> courses = new HashSet<>();

        courses.add("Java");
        courses.add("Python");
        courses.add("C");
        courses.add("Java");

        System.out.println("After add(): " + courses);

        courses.remove("C");
        System.out.println("After remove(): " + courses);

        System.out.println("Contains Python? "
            + courses.contains("Python"));
        System.out.println("Set size: " + courses.size());
        System.out.println("Is set empty? " + courses.isEmpty());

        courses.clear();
        System.out.println("After clear(): " + courses);
        System.out.println("Is set empty after clear()? "
            + courses.isEmpty());
    }
}

```

6. Completion and Execution Commands

Step 1: Save the program as:
SetMethod.java

Step 2: Compile the program:
javac SetMethod.java

Step 3: Run the program:
java SetMethod

7. Output

```

After add(): [Java, C, Python]
After remove(): [Java, Python]
Contains Python? true
Set size: 2
Is set empty? false
After clear(): []
Is set empty after clear()? true

```

Note: HashSet does not guarantee iteration order; the displayed order may vary.

Result: The Java program was successfully implemented and the specified collection framework methods were demonstrated.

Experiment No: 11

Basic Information

Experiment No	11
Date	21/09/26
Title	Implementation of SortedMap Interface Methods in Java
Name	S.Trivikram
Roll Number	A25126552251

1. Objective / Aim

To implement and demonstrate SortedMap methods firstKey(), lastKey(), headMap(), tailMap(), subMap(), and comparator() using TreeMap.

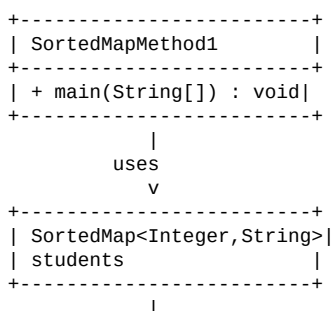
2. System Requirements

Hardware Requirement	Specification
Processor	Intel Core i3 or equivalent and above
RAM	Minimum 4 GB
Storage	Minimum 1 GB free space
Input Device	Keyboard and mouse
Display	Standard monitor
Software Requirement	Specification
Operating System	Windows / Linux / macOS
Java	JDK 17 or JDK 21
IDE / Editor	Eclipse / IntelliJ IDEA / NetBeans / VS Code
Compiler	javac
Execution Environment	Java Virtual Machine (JVM)

3. Algorithm / Logic Description

1. Start the program.
2. Import SortedMap and TreeMap.
3. Create a SortedMap of student IDs and names.
4. Insert key-value pairs using put().
5. Display the sorted map.
6. Find the first and last keys using firstKey() and lastKey().
7. Create portions using headMap(), tailMap(), and subMap().
8. Display comparator() and stop the program.

4. UML Class Diagram



```

        implemented by
          v
+-----+
| TreeMap<Integer,String> |
+-----+

```

5. Source Code

```

import java.util.SortedMap;
import java.util.TreeMap;

class SortedMapMethod1 {
    public static void main(String[] args) {
        SortedMap<Integer, String> students = new TreeMap<>();

        students.put(103, "Rohan");
        students.put(101, "Aarav");
        students.put(105, "Diya");
        students.put(102, "Meera");
        students.put(104, "Anjali");

        System.out.println("SortedMap: " + students);
        System.out.println("First Key: " + students.firstKey());
        System.out.println("Last Key: " + students.lastKey());
        System.out.println("HeadMap (< 103): "
            + students.headMap(103));
        System.out.println("TailMap (>= 103): "
            + students.tailMap(103));
        System.out.println("SubMap (102 to 105): "
            + students.subMap(102, 105));
        System.out.println("Comparator: " + students.comparator());
    }
}

```

6. Completion and Execution Commands

Step 1: Save the program as:
SortedMapMethod1.java

Step 2: Compile the program:
javac SortedMapMethod1.java

Step 3: Run the program:
java SortedMapMethod1

7. Output

```

SortedMap: {101=Aarav, 102=Meera, 103=Rohan, 104=Anjali, 105=Diya}
First Key: 101
Last Key: 105
HeadMap (< 103): {101=Aarav, 102=Meera}
TailMap (>= 103): {103=Rohan, 104=Anjali, 105=Diya}
SubMap (102 to 105): {102=Meera, 103=Rohan, 104=Anjali}
Comparator: null

```

Result: The Java program was successfully implemented and the specified collection framework methods were demonstrated.

Experiment No: 12

Basic Information

Experiment No	12
Date	21/09/26
Title	Implementation of SortedSet Interface Methods in Java
Name	S.Trivikram
Roll Number	A25126552251

1. Objective / Aim

To implement and demonstrate SortedSet methods first(), last(), headSet(), tailSet(), subSet(), and comparator() using TreeSet.

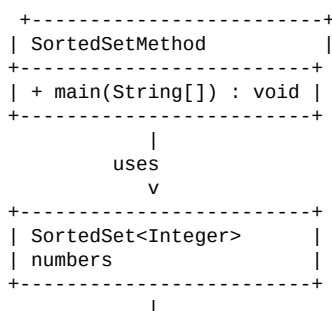
2. System Requirements

Hardware Requirement	Specification
Processor	Intel Core i3 or equivalent and above
RAM	Minimum 4 GB
Storage	Minimum 1 GB free space
Input Device	Keyboard and mouse
Display	Standard monitor
Software Requirement	Specification
Operating System	Windows / Linux / macOS
Java	JDK 17 or JDK 21
IDE / Editor	Eclipse / IntelliJ IDEA / NetBeans / VS Code
Compiler	javac
Execution Environment	Java Virtual Machine (JVM)

3. Algorithm / Logic Description

1. Start the program.
2. Import SortedSet and TreeSet.
3. Create a SortedSet of integer values.
4. Add the given values; TreeSet maintains them in sorted order.
5. Display the sorted set.
6. Find the first and last elements using first() and last().
7. Create subsets using headSet(), tailSet(), and subSet().
8. Display comparator() and stop the program.

4. UML Class Diagram



```

        implemented by
          v
+-----+
| TreeSet<Integer> |
+-----+

```

5. Source Code

```

import java.util.SortedSet;
import java.util.TreeSet;

class SortedSetMethod {
    public static void main(String[] args) {
        SortedSet<Integer> numbers = new TreeSet<>();

        numbers.add(40);
        numbers.add(10);
        numbers.add(70);
        numbers.add(20);
        numbers.add(50);
        numbers.add(30);

        System.out.println("SortedSet: " + numbers);
        System.out.println("First element: " + numbers.first());
        System.out.println("Last element: " + numbers.last());
        System.out.println("HeadSet before 40: "
            + numbers.headSet(40));
        System.out.println("TailSet from 30: "
            + numbers.tailSet(30));
        System.out.println("SubSet from 20 to 60: "
            + numbers.subSet(20, 60));
        System.out.println("Comparator: " + numbers.comparator());
    }
}

```

6. Completion and Execution Commands

Step 1: Save the program as:
SortedSetMethod.java

Step 2: Compile the program:
javac SortedSetMethod.java

Step 3: Run the program:
java SortedSetMethod

7. Output

```

SortedSet: [10, 20, 30, 40, 50, 70]
First element: 10
Last element: 70
HeadSet before 40: [10, 20, 30]
TailSet from 30: [30, 40, 50, 70]
SubSet from 20 to 60: [20, 30, 40, 50]
Comparator: null

```

Result: The Java program was successfully implemented and the specified collection framework methods were demonstrated.